Paper 3 — Isaac Sim Tactile Data Factory

Handoff v4 | Kourosh (PhD, ETS CoRo Lab) | UR5e + Robotiq 2F-85 + two TSF-85 tactile sensors
Prepared to start a fresh chat with full continuity. Read this top to bottom before continuing.

What this project is. Build a data-collection "factory" in Isaac Sim 5.1 that grasps an object with two deformable TSF-85 tactile pads at many designed positions (a GUI-defined grid), records the temporal tactile signal at each grasp, and later stitches these into a large contact map. This feeds Paper 3, whose contribution is replacing the hand-crafted shape-classifier extrapolation of Papers 1-2 with a model learned on large-scale simulated tactile data (inspired by Roberge et al. 2021 CASE, whose 97.7%-accuracy modality is the perceived deformation during compression, sampled at 5/50/95% of max).

Bottom line of this session. The pad-placement / calibration problem is now SOLVED and verified. A normal grasp lands the pad centre on the GUI-designed target to **+0.2 mm**, pads symmetric to 0.01 mm, with the pad pose read live from physics. Everything below is the how, the why, the constants, and what's left.

Item	Status
Physics explosion at gripper close	FIXED (root cause found)
Jerky / lift-to-home motion between grid points	FIXED (pad-to-pad)
Pad placed wrong vs GUI (the big one)	FIXED + verified +0.2 mm
Live pad-pose reading (Case prims)	WORKING (is_live True)
Calibration = measured, per-diameter, auto-applied	DONE
Contact gate (refuse air grasps)	DONE (tactile_peak_sum)
Reachability pre-check + JSON log + colored grid	DONE
GUI visuals (measured overlay, flange/palm, Reset)	DONE
Full multi-point (2x3) grid verification	PENDING (running now)
Contact-aware close (for OTHER diameters)	PARKED (needed later)
Block 2: stitching tactile maps	LATER (not started here)

1. Scene, hardware & verified geometry

All numbers below were MEASURED from the USD or from live probes this session — not assumed.

Quantity	Value (mm unless noted)
Cylinder (object)	diameter 26.0, length 140.0
Cylinder centre (world)	[-268.06, 199.0, 1052.2]
Cylinder span (world Z)	982.2 (bottom) to 1122.2 (top)
Robot base_link (world)	[20.93, -337.5, 992.75]
Table top (world Z)	~985
EE (tool0) -> gripper palm (base_link)	10.79 mm down the tool axis
EE -> pad CENTRE (calibrated, 26mm)	155.16 mm (TOOL_OFFSET_Z=0.15671)
Pad face size (from node cloud)	~41 long (up rod) x 26 x ~3 thick
Case-origin -> pad centre (up rod)	22.3 mm (PAD_CENTER_ABOVE_CASE_M=0.0223)
Gripper close command	CLOSE_RAD = 0.55 rad (of ~0.82 max)
Squeeze axis	world X (two pads meet along X)
Pad long axis / rod axis	world Z

Two-Python wall. The GUI (matplotlib/tkinter) and the Isaac collector run as SEPARATE processes. The GUI never launches Isaac directly; it saves a config JSON and shows a terminal command you copy-run. Keep this separation.

Coordinate frames — critical gotcha. UsdGeom.XformCache and usdrt both return FROZEN/authored or no-world poses for the sensor housing top-level Xform. The REAL live sensor body is the prim named **Case**, two levels deep (.../TSF_85_right/TSF_85/TSF_85/Case). We search the stage for prims named 'Case' under TSF_85 rather than hard-coding the path (depth differs between raw USD and referenced stage).

2. Key files & what they do

Path	Role
~/Paper3_Simulation/sim/collect_from_config.py	Isaac-side collector: motion (cuRobo), grasp, close, tactile record, pad-truth probe, calibr
~/Paper3_Simulation/.../main_gui.py	Design GUI: grid design, preview (TOP/FRONT/3D), Calibrate tab, reachability buttons, p
~/Paper3_Simulation/TSF-85/TSF_85_Ext/.../extension.py	Berith's tactile extension. Writes BASENAME_s1/s2_tactile_maps.csv and _mesh_state.
~/Paper3_Simulation/Data/pad_offset_calibration.json	Calibration store, keyed by diameter. Normal runs read this; REFUSE if uncalibrated.
~/Paper3_Simulation/Data/gui_config.json / gui_calib_config.json	Config the GUI writes for a normal run / a calibrate run.
~/Paper3_Simulation/curobo-stable/src	cuRobo source, injected via sys.path (NOT pip-installed).
scenes/ur5e.yml	cuRobo robot config. Arm collision spheres only; NO gripper/finger spheres.
assets/Robotiq_2F_85_modified.usd	The modified gripper (adapter + TSF pads).

3. Every problem this session: symptom, root cause, fix

3.1 Physics explosion + sensor pads fall off, right before grasp

- **Symptom:** robot spins wildly, articulation goes NaN, deformable pads fly to z=-18 m at gripper close.
- **False leads (ruled out by test):** a cuRobo collision-world (table slab) I added — reverting it did NOT fix it; and GPU-dynamics/deformable setup (identical to Berith's stable baseline).
- **Root cause:** the pad-truth probe constructed SingleRigidPrim / SingleXFormPrim on LIVE articulation finger links mid-simulation. Wrapping a running articulation link in a fresh physics view corrupts PhysX -> NaN at close.
- **Fix:** read poses only via PASSIVE readers (UsdGeom.XformCache + usdrt); NEVER construct SingleRigidPrim/SingleXFormPrim on live links. Explosion gone; 100+ grasps stable since.

3.2 Robot lifts 100 mm and swings toward home between every grid point (jerky)

- **Root cause:** grasp_one_point lifted APPROACH_H (100 mm) + did a GLOBAL cuRobo replan for each point, with no seed continuity -> wildly different joint branch each hop.
- **Fix:** POINT_TO_POINT=True. First point uses the proven approach (free-move to UP + stitched descent); later points move DIRECTLY pad-to-pad at grasp height via plan_stitched_line, seeded from the previous joints. Retreat only on the last point. Collision-safe because the grid keeps X fixed (pads move only in world Y-Z).

3.3 Gripper tilted ~2 deg / one pad crushes, the other barely touches (s1=20023 vs s2=276)

- **Two bugs, found separately:**
- (a) plan_stitched_z (descent) and plan_stitched_line targeted the DRIFTING current quaternion (cquat) instead of the FIXED tool-down quaternion (tq). Each step inherited the previous step's orientation error -> tilt compounded. Fix: both target tq every step.
- (b) Only finger_joint was driven; the RIGHT 4-bar (right_outer_knuckle_joint) followed passively through a compliant PhysX loop and settled at a different angle -> 0.9 deg finger mismatch -> pads unequal. Fix: DRIVE_BOTH_FINGERS=True commands BOTH drive joints to the same angle.
- **Result:** pads now 0.01 mm apart, fingers 0.00 deg apart.

3.4 The '16.46 mm / 2.04 deg convergence bug' was NOT a planner bug

- **Symptom:** a CENTER grasp landed the EE 16.46 mm too high and 2.04 deg tilted.
- **Root cause:** PALM STRIKE. Grasping the CENTRE of a 140 mm rod leaves 70 mm of rod poking UP into the finger linkage (inner-finger joint at 1114.9 < rod top 1122.2). The rod physically blocks the descent and levers the arm off-axis.
- **Proof:** at the TIP (z-offset ~+70), the same command lands to 0.09 mm and 0.005 deg. Perfect.
- **Rule:** NEVER grasp the centre of the 140 mm rod. Stay in the collision-free band (pad Z offset ~+40 to +70). This is a grasp-geometry constraint, not a bug to fix.

3.5 cuRobo 20-hour 'compile' hang (twice)

- **Root cause:** (1) 'ninja' was NOT installed -> single-threaded JIT compile of CUDA kernels = ~20 h instead of ~1 min. (2) A Ctrl+C during compile left a STALE LOCK that froze every later run at a black window.

- **Fix:** ~/isaacsim/python.sh -m pip install ninja (1.13.0). Delete stale lock: rm -f ~/.cache/torch_extensions/py311_cu128/*/lock.
- **Rules:** NEVER Ctrl+C during 'JIT compiling...'. Copying/moving the project folder refreshes source mtimes -> triggers a full rebuild (use cp -a / rsync -a to preserve mtimes). Prefix runs with TORCH_CUDA_ARCH_LIST="7.5" (RTX 2060 = sm_75). Kernels cache in ~/.cache/torch_extensions; curobo itself is NOT pip-installed (sys.path injected in collect_from_config.py line ~16).
- **Diagnose a hang:** check for a 'lock' file and run 'pgrep -c nvcc'. Zero nvcc + lock present = hung, not compiling.

4. The calibration system (the central achievement)

4.1 The old broken model

- TOOL_OFFSET_Z was a hand-tuned SCALAR (0.158) 'EE sits this far straight above the pad'. It silently assumes (a) the tool is perfectly vertical and (b) both pads are identical - both were false.
- pad_pose_from_joints() FK used a guessed constant PAD_OFF_WRIST and was ~60 mm off in X - it is UNUSED for planning (logging only). Do not trust pad_actual_fk_m.
- A fudge constant C_ANCHOR=0.1332 patched the scalar. All of this is now obsolete.

4.2 The correct model - measured live pad pose

We read the LIVE world pose of each sensor Case prim at closed grip (proven live: pads move mirror-symmetric [-28,0,-13] / [+28,0,-13] mm open->closed = squeeze + swing). The offset is then MEASURED, not derived:

```
grasp_centre = midpoint(pad_right_Case, pad_left_Case) # world, at closure
offset_world = grasp_centre - EE_world # measured 3-vector
TOOL_OFFSET_Z_case_origin = -offset_world.z # ~0.13441 m
TOOL_OFFSET_Z = case_origin + PAD_CENTER_ABOVE_CASE_M # ~0.15671 m (pad CENTRE)
```

- The Case ORIGIN sits at the pad's END, ~22.3 mm from the pad CENTRE along the rod. The GUI draws the pad CENTRED on the target, so calibration adds PAD_CENTER_ABOVE_CASE_M=0.0223 to target the CENTRE, not the edge.
- For a symmetric tool-down grasp the x,y of the midpoint offset are ~0, so commanding still uses the scalar TOOL_OFFSET_Z (EE = pad_target + [0,0,TOOL_OFFSET_Z]) - but the number is now MEASURED at a clean grasp, per diameter.
- Stored per diameter in pad_offset_calibration.json. Normal runs look up by diameter and REFUSE if uncalibrated (env GRASP_TOOL_OFFSET= overrides for dev).

4.3 Contact gate (added late in session)

- Problem: a calibrate grasp that closes on AIR (e.g. pad Z above the rod tip) still reads 'is_live' True (fingers swing in air) and was SILENTLY storing a garbage offset.
- Fix: before storing, read the pt00 tactile PEAK sum for both sensors. Baseline ~250, real contact ~6000-12000. If peak < CONTACT_MIN_SUM (1000), RAISE _CalNoContact and REFUSE to store (prints 'REFUSING TO STORE', keeps previous calibration). Logs tactile_peak_sum in the entry.
- Verified: a real z=40 grasp logged tactile_peak_sum=12672.4. Note the refusal only prints to the TERMINAL, not the GUI.

4.4 The verification trap that cost us a day (READ THIS)

- The GUI 'Last grasp landing' error reads the newest pad_truth_probe.json. BUT a CALIBRATE run uses a PROVISIONAL offset (CAL_DEFAULT_OFFST=0.15), NOT the stored calibrated value. So a calibrate-run's landing error is MEANINGLESS for judging placement.
- This made trimming PAD_CENTER_ABOVE_CASE_M move the error the wrong way and led to a false '-7 mm constant error'.
- CORRECT TEST: run a NORMAL collection grasp (which uses the stored offset), then read THAT run's pad_truth_probe.json. Result: pad centre landed 1092.4 vs target 1092.2 = +0.2 mm. Calibration is correct; 0.0223 is the right value.

5. Reachability pre-check (ported from Paper 2)

- Before any motion, every grid point is dry-run: IK at its EE target, then simulate the incremental trajectory and enforce Paper-2 gates. Nothing moves. Writes reachability_report.json; unreachable points are SKIPPED and logged, never fatal.
- Kept from Paper 2 verbatim: incremental dry-run (steps = ceil(|dq|/0.02), capped 500), gates (frozen_joints, one_sided, delta_bounds, per_iter_dq_cap, abs_limits), cond(J)>1e3 rejection, 1mm/0.5deg final FK tolerance, {reachable,reason,limit_hits} contract.
- Swapped for this pipeline: MoveIt FK -> cuRobo fk(q); real Jacobian -> _numeric_jacobian6 (finite diff on FK). FIXED a Paper-2 bug: J is now evaluated AT each trajectory q (was current-q only), so cond(J) is meaningful.
- MANUAL_LIMITS default PERMISSIVE (gates off, cond(J) on, abs_limits=None) - Paper-2's tight limits were for a local regrasp SEARCH; a grid scan needs the arm free to travel. No GUI tab needed.
- Env: GRASP_REACH_ONLY=1 (check + exit, no motion), GRASP_REACH_CHECK (auto-check before a run), GRASP_REACH_SKIP.
- GUI: 'Check Reachability (no motion)' + 'Load reachability result' buttons; grid dots colored green (reachable) / red dashed (unreachable).

6. GUI state (main_gui.py)

- Tabs: Collection, Calibrate, Stitching (Block 2).
- Calibrate tab: object+pad FRONT preview; Z-offset control (Y locked to centre - off-centre Y would grip a chord < diameter and give a wrong number); Run-headless toggle; 'Save + Show Calibrate Command'; 'Load calibration result' (full readout: method, case-origin, pad-centre shift, L/R symmetry, last-grasp landing); 'Reset view' button; green dashed 'measured pad' overlay gated behind _show_measured (default False -> clean plot on open).
- Collection FRONT frame: draws EE (flange) marker + palm marker below it, with EE-to-pad and palm-to-rodtop clearance annotations (calibration made visible).
- Helpers: _cal_entry() reads pad_offset_calibration.json for the current diameter; _newest_probe() reads the newest pad_truth_probe.json.
- All additive blocks are guarded with try/except so missing data can never break the GUI.

7. Canonical constants (current, verified)

Constant	Value	Meaning
TOOL_OFFSET_Z (stored, 26mm)	0.15671 m	EE above pad CENTRE, measured
TOOL_OFFSET_Z_case_origin	0.13441 m	EE above Case origin
PAD_CENTER_ABOVE_CASE_M	0.0223 m	Case origin -> pad centre (KEEP; 0.0293 was WRONG)
C_ANCHOR	0.1332 m	obsolete fallback only
CAL_DEFAULT_OFFST	0.15 m	provisional offset during a calibrate grasp
CLOSE_RAD	0.55 rad	gripper close command (object may block early)
CONTACT_MIN_SUM	1000	tactile peak gate (baseline ~250, contact ~6000)
APPROACH_H	0.10 m	lift height for first-point approach
cond(J) reject / step / cap	1e3 / 0.02 / 500	reachability gates
EE -> palm	10.79 mm	tool0 to base_link, down tool axis

TORCH_CUDA_ARCH_LIST	7.5	RTX 2060 = sm_75 (prefix all runs)

8. Hard-won rules (do not relearn these)

- NEVER construct SingleRigidPrim/SingleXFormPrim on live articulation links -> physics explosion. Use passive XformCache / usdrt only.
- The live sensor body is the 'Case' prim two levels deep - SEARCH for it, do not hard-code the path. The top-level TSF_85 Xform is frozen (empty wrapper).
- NEVER grasp the centre of the 140 mm rod (palm/finger strike). Stay pad Z-offset ~+40..+70.
- A CALIBRATE run's landing error is meaningless (provisional offset). Judge placement only from a NORMAL run.
- Calibrate ONLY where the pads actually contact the object (Y centred, Z on the body). The contact gate now enforces this.
- NEVER Ctrl+C during cuRobo JIT compile (stale lock -> permanent hang). Keep ninja installed. Copy folders with cp -a/rsync -a.
- 0.55 close is a COMMAND, not the achieved angle - the object blocks the fingers early; the achieved angle is object-dependent.
- When Kourosh's visual read of the sim disagrees with a diagnostic, Kourosh has been right every time - measure, do not dismiss.
- Both GUI + collector must be edited on the CURRENT running files. File drift caused several bugs this session; always verify with grep before editing (e.g. grep -c measured_live_pad, _find_case_prims).

9. Next steps (in order)

- **NOW (in progress):** run a 2x3 grid at a safe Z band (e.g. z=40, on the rod body). Confirm EVERY point lands on target (pad centre error ~0), pads symmetric (Z apart ~0), tactile is real (peak >> 1000) and s1 ~ s2. Paste pose_history.json + pad_truth_probe.json to verify each point.
- **THEN:** scale to larger grids (e.g. 100-point) only after 2x3 is clean.
- **Contact-aware close (still PARKED, needed for OTHER diameters):** 0.55 only reaches the 26 mm rod; a thinner object never gets touched, a fatter one is crushed (the 0.8 test collapsed the signal 17900 -> 1500). Plan: close until tactile sum crosses ~1000 (4x baseline) + margin, with a hard-angle safety cap. Reads SensorChannel.last_pred. Required to reproduce Roberge's 5/50/95%-of-max deformation stages. NOTE: this does NOT fix pad-centering (a separate, solved problem) - it only makes one setting work across diameters.
- **Block 2 - stitching:** assemble per-grasp tactile maps into a world-frame contact map (viz/stitching.py exists from earlier work). Uses the now-correct pad poses.
- **Data-quality watch:** at the rod tip (z~70) part of the pad hangs off the end -> partial contact. For real data use z that keeps the whole pad on the body.

10. What to give the new chat to resume instantly

- This PDF.
- Current collect_from_config.py and main_gui.py (the RUNNING ones - verify with: grep -c measured_live_pad collect_from_config.py).
- pad_offset_calibration.json (the calibration store).
- The newest pose_history.json + pad_truth_probe.json from the 2x3 run.
- extension.py (Berith's) + one pair of *_mesh_state.csv if touching stitching.

- Berith's touch_cylinder.py and the Roberge 2021 CASE PDF if revisiting the close / temporal stages.

Prepared at the end of a long multi-session debugging effort. The calibration problem - pad placement not matching the GUI - is solved and verified to +0.2 mm. The immediate task is confirming a full 2x3 grid, then contact-aware close for other diameters, then Block 2 stitching.